

LEGALEASE

AI-Powered Legal Document Generator

PROJECT DOCUMENTATION

Prepared from the supplied LegalEase project source code
and the supplied LegalEase reference PDF

September 2026

1. Abstract

LegalEase is a Streamlit and FastAPI web application for generating editable legal-document templates with Google's Gemini API. The system accepts a document type, parties, terms and an effective date, sends those inputs to the backend AI layer, displays the generated document for editing, and provides TXT, DOCX and PDF export options. The reference material describes use cases including employment contracts, non-disclosure agreements and lease agreements, while the supplied implementation provides the corresponding user interface, API, AI integration and export utilities.

2. Project Objectives

- Provide an accessible interface for drafting structured legal-document templates.
- Generate document text from user-supplied facts without inventing missing facts.
- Allow users to review and edit generated text before export.
- Export the result as plain text, Microsoft Word DOCX, or formatted PDF.
- Support optional branding through a user-uploaded logo.
- Keep the application modular through separate frontend, backend, AI and utility components.

3. Scope and Use Cases

The reference PDF identifies the main scenarios as employment contracts, NDAs and residential lease agreements. The implemented Streamlit interface additionally offers Service Agreement, Freelance Work Contract, Employment Offer Letter, General Agreement and Custom Legal Document options.

Typical workflow

1. Select the required document type.
2. Enter the parties involved.
3. Enter terms and conditions separated by semicolons.
4. Select the effective date.
5. Send the request to the FastAPI backend.
6. Generate a structured template through Gemini.
7. Edit and preview the returned document.
8. Download the final text as TXT, DOCX or PDF.

4. System Architecture

The supplied project follows a layered architecture. Streamlit provides the presentation layer; FastAPI provides the HTTP API; GeminiDocumentGenerator handles AI generation; and utility modules sanitize text and create output files.

Layer	Component	Responsibility
Frontend	app.py / Streamlit	Input collection, preview, editing, branding upload and downloads
Backend	backend/main.py	FastAPI application, health endpoints and router registration
API	backend/routes.py	Pydantic validation and POST /generate request handling
AI Core	ai_core/gemini_generator.py	Prompt construction and Gemini content generation

Utilities	utils/sanitize.py	Text normalization and term splitting
Utilities	utils/exporters.py	TXT, DOCX and PDF generation
Testing	tests/	API and sanitization tests

5. Project Structure

```

LegalEase/
├── ai_core/
│   └── gemini_generator.py
├── backend/
│   ├── main.py
│   └── routes.py
├── utils/
│   ├── exporters.py
│   └── sanitize.py
├── tests/
│   ├── test_api.py
│   └── test_sanitize.py
├── app.py
├── .env.example
├── requirements.txt
├── Dockerfile
├── Procfile
├── run_backend.bat
├── run_frontend.bat
└── SETUP_QUICK.txt

```

6. Frontend Module

6.1 User Interface

The Streamlit application uses a wide layout with a settings sidebar and two main columns. The sidebar accepts the FastAPI backend URL and an optional PNG/JPG/JPEG logo. The main interface collects document details and displays the generated result in an editable text area and a styled preview.

6.2 Input Fields

- Document Type: selected from predefined legal-document categories or a custom type.
- Parties Involved: names and roles of the participants.
- Terms & Conditions: user-provided clauses separated by semicolons.
- Effective Date: selected using Streamlit's date input.

6.3 Output

- Editable document text
- HTML-style preview
- TXT download
- DOCX download with optional logo and key-terms table

- PDF download with header/footer and optional logo

7. Backend API

7.1 FastAPI Application

backend/main.py initializes the FastAPI application, configures permissive CORS for the current implementation, includes the document router, and exposes / and /health endpoints.

7.2 POST /generate

backend/routes.py defines DocumentRequest with validation for document_type, parties, terms and effective_date. The endpoint creates a GeminiDocumentGenerator instance, generates the document and returns a DocumentResponse containing document_type and content.

Field	Purpose
document_type	Type of legal document
parties	People or organizations involved
terms	Terms and conditions supplied by the user
effective_date	Date the document takes effect

8. AI Generation Module

ai_core/gemini_generator.py uses the current Google GenAI Python SDK. The model is configurable through GEMINI_MODEL, with the project README specifying gemini-3.8-flash as the default implementation model. The generator builds a structured prompt and applies a system instruction requiring a professional template, preservation of supplied facts, neutral legal language, placeholders for missing critical facts, numbered sections and a final review notice.

Generation safeguards implemented

- Do not invent names, dates, addresses, prices, obligations, laws, statutes or other facts.
- Use [TO BE COMPLETED] when a critical fact is missing.
- Do not present the generated template as universally legally valid.
- Use neutral and formal legal language.
- Tell users to obtain qualified legal review before signing or relying on the document.

The generator retries selected temporary failures (HTTP-like 429, 500 and 503 codes) up to three attempts with exponential delays.

9. Document Export and Formatting

9.1 Text sanitization

sanitize.py normalizes line endings, converts typographic punctuation to ASCII-friendly characters, collapses repeated spaces and excessive blank lines, and provides split_terms() for semicolon-separated clauses.

9.2 DOCX

The DOCX exporter uses python-docx. It applies Times New Roman formatting, creates a centered document header, optionally embeds a logo, detects headings and numbered clauses, formats bullet lines, creates a KEY TERMS table from semicolon-separated input, and adds a footer containing the LegalEase review notice.

9.3 PDF

The PDF exporter uses FPDF. It provides a centered document-type header, optional logo placement, automatic page breaks, heading detection, and a footer containing the LegalEase name, page number and review notice.

9.4 TXT

The TXT exporter returns the sanitized document text as UTF-8 bytes.

10. Security, Privacy and Legal Considerations

- The application loads the Gemini API key from environment variables rather than hard-coding it.
- The README advises not committing .env to Git.
- The current backend uses `allow_origins=['*']`; production deployment should restrict CORS to the actual frontend domain.
- The README recommends authentication and rate limiting for production.
- Sensitive document contents should not be logged unnecessarily.
- HTTPS should be used for production deployments.
- LegalEase is a document-template generator and is not a substitute for advice from a qualified lawyer.

11. Installation and Configuration

9. Install Python 3.10 or newer; the README recommends Python 3.12.
10. Open the LegalEase folder in VS Code.
11. Create a virtual environment with `python -m venv .venv`.
12. Activate the environment.
13. Install dependencies using `pip install -r requirements.txt`.
14. Copy `.env.example` to `.env`.
15. Set `GEMINI_API_KEY` and optionally `GEMINI_MODEL` and `BACKEND_URL`.

The supplied requirements include FastAPI, Uvicorn, Streamlit, Requests, python-dotenv, Google GenAI, python-docx, fpdf2, Pillow, pytest and httpx.

12. Running the Application

Backend

Run: `uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000`

The root service is available at `http://127.0.0.1:8000` and FastAPI's interactive documentation at `/docs`.

Frontend

Run: `streamlit run app.py`

Streamlit normally starts at `http://localhost:8501`.

Automated tests

Run: `pytest -q`

The supplied API test mocks Gemini generation, so it does not consume an API quota.

13. Testing and Verification

- Health endpoint verification through GET /health.
- Document generation API verification using a mocked Gemini generator.
- Text sanitization verification for punctuation and blank-line normalization.
- Semicolon-separated term parsing verification.
- Manual end-to-end verification: generate, edit, preview and download TXT/DOCX/PDF.
- Formatting checks should include logo placement, headings, footer content, bullet points and key-term tables.

14. Deployment

The project contains both a Dockerfile and a Procfile. The Dockerfile uses python:3.12-slim, installs requirements, copies the application and runs Uvicorn on port 8000. The Procfile starts the FastAPI service using the platform-provided PORT.

The reference document also discusses deployment options including Render, Railway, Deta, Streamlit Community Cloud, Fly.io and VPS hosting. These are presented as possible deployment targets rather than verified deployments of the supplied project.

15. Limitations and Future Enhancements

- Legal validity depends on jurisdiction and factual completeness; generated templates require review.
- Production CORS configuration should be restricted.
- Authentication, rate limiting and stronger operational security can be added.
- Future enhancements described in the reference include deeper contract analysis, legal-database integration and personalized recommendations.
- The reference also describes multilingual capabilities as a broader platform direction; the supplied code does not show a dedicated language-selection feature.

16. Conclusion

LegalEase combines a Streamlit user interface, FastAPI backend, Google Gemini generation, text sanitization and multi-format export into a single legal-document drafting workflow. The supplied implementation is organized into clear modules and includes automated tests, local-run scripts, Docker support and production-oriented configuration guidance. Its central design principle is to generate editable templates from user-provided facts while explicitly avoiding unsupported legal claims and directing users toward professional review.

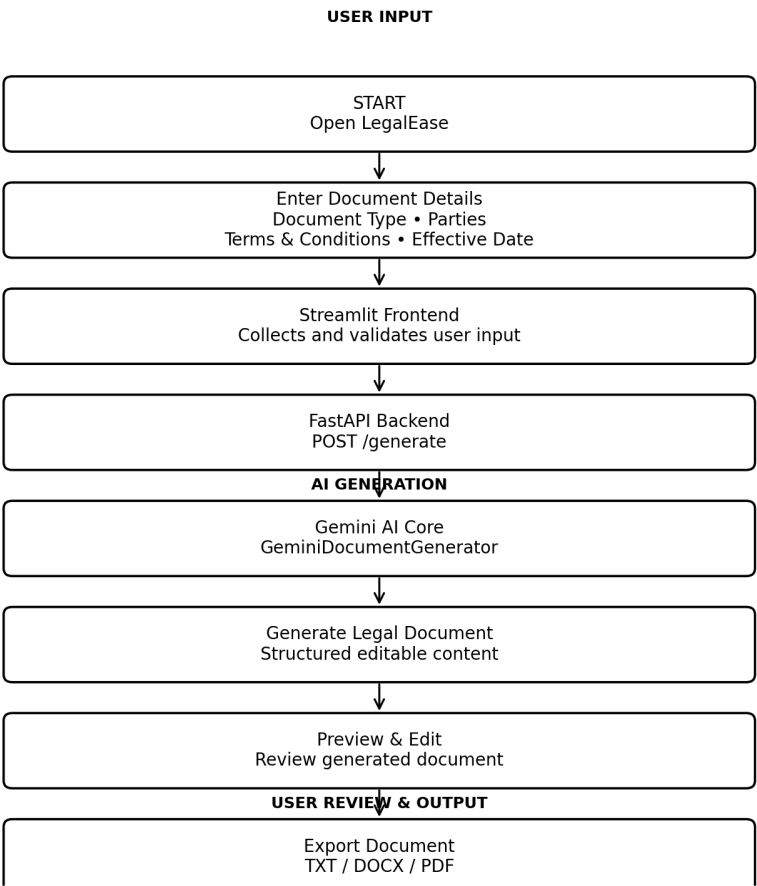
17. Reference Basis

This documentation was prepared by inspecting the supplied project archive and using the supplied LegalEase PDF as the reference document. The reference PDF describes the project purpose, scenarios, prerequisites,

architecture, milestones, generation workflow, export formats, deployment process and conclusion. The project archive was used to document the actual implementation details, including current file structure, API fields, SDK usage, model configuration, testing and export behavior.

18. Project Workflow Flowchart

The workflow below follows the project flow described in the supplied reference PDF: user input is collected through Streamlit, sent to the FastAPI /generate endpoint, processed by the Gemini document generator, returned for preview/editing, and exported as TXT, DOCX or PDF.



Submitted by

Team Members

G.Kiridharan

S.Sivanesh

K.Kishanth

V.Madho

Mentor: Krishnan N

Date: 24/09/2026